

Zero-Error Recovery from Deterministic Partial Views

Tristan Simas^a

^a*McGill University, 845 Sherbrooke Street West, Montreal, QC H3A 0G4, Canada*

Abstract

We study zero-error recovery from deterministic partial views of a finite latent tuple. Admissible views induce a confusability graph on latent states, reducing exact recovery with a T -ary auxiliary tag to T -colorability. In the full coordinate-view model on the labeled tuple space, we characterize the realizable confusability relations exactly as those determined by upward-closed families of coordinate-agreement sets. Under repeated composition, the block confusability graph is the strong power of the one-shot graph, so the normalized zero-error rates converge to the Shannon capacity of the induced graph and inherit the standard Lovász- ϑ upper theory. Transitive confusability yields a cluster-graph equality route, with meet-witnessing and fiber coherence as sufficient conditions. Under an affine restriction on the realized state family, the coordinate side carries a representable matroid whose rank gives tractable upper bounds on confusability and capacity and composes by direct sum under block composition. Applied to representative channel families, the realizability criterion classifies eleven deterministic partial-view architectures and finds nine operating outside the verifiable zero-error regime.

Keywords: zero-error recovery, confusability graphs, strong graph powers, Shannon capacity, side information, matroids, formal verification

1. Introduction

We study exact recovery from a finite latent tuple observed through deterministic partial views. A family of admissible views induces a confusability graph on latent states: two states are adjacent when some allowed view fails to separate them. Exact zero-error recovery with an auxiliary tag is therefore

* Corresponding author
Simas)

Email address: `tristan.simas@mail.mcgill.ca` (Tristan

a graph-coloring problem, and the one-shot converse gives the finite obstruction on which the later graph-capacity theory is built.¹

In the exact coordinate-view model on the full labeled tuple space, confusability depends only on coordinate-agreement sets. This yields a realizability theorem: the confusability relations generated by admissible coordinate-subset views are exactly those determined by upward-closed families of agreement sets.²

The binary two-factor square establishes that the induced graph need not be a clique: with single-coordinate views the four latent states form a 4-cycle, so exact recovery needs two tags rather than four. Repeated composition preserves that induced graph structure through strong powers, placing the model inside classical zero-error graph-capacity theory [1–8].³

Contributions. First, admissible partial views induce a confusability graph on latent tuples, and exact recovery with a T -ary auxiliary tag is equivalent to T -colorability of that graph. Second, in the full coordinate-view model, the realizable confusability relations are exactly the upward-closed agreement-set families. Third, block composition maps the one-shot confusability graph to its strong powers. Fourth, the normalized block rates converge to the Shannon capacity of the induced graph and inherit standard complement-chromatic and Lovász- ϑ upper bounds. Fifth, transitivity of confusability yields a cluster-graph equality route, with meet-witnessing and fiber coherence as sufficient conditions. Sixth, under an affine restriction on the realized state family, the coordinate side carries a representable matroid whose rank gives tractable upper bounds on confusability and capacity.⁴

Organization. Section 2 sets up the model and the zero-incoherence threshold. Section 3 develops the finite converse foundation. Sections 4–7 contain the graph characterization, strong-power composition, asymptotic capacity, theta upper theory, and equality route. Section 8 gives the affine matroid specialization. Sections 9–11 summarize the unit-rate boundary and realizability consequences in the main text. Appendix A contains the detailed boundary and realizability proof package, and Appendix B records representative model instantiations. A supplementary Lean 4 artifact

¹ Lean: OBS1–2, OBS5, CIA3, MFT3 ² : MFT118–131 ³ : MFT5–10, MFT24, MFT35 ⁴ : MFT3, MFT20, MFT35, MFT48, MFT69, MFT96, MFT102, MFT109, AFM1–11

provides a machine-checked verification of the core theorem chain, including the capacity bounds and the equality-route implications [9, 10].

2. Model and Basic Quantities

2.1. Model assumptions and notation

We use the following standing assumptions.

1. **Finite latent state:** each fact takes values in a finite alphabet.
2. **Deterministic observations:** a system state exposes a deterministic observation transcript.
3. **Independent rate:** the independent rate counts the number of independently writable source locations for the same latent fact.
4. **Finite side information:** any auxiliary tag is drawn from a finite alphabet.

These are the only ingredients needed for the deterministic zero-error theory developed below.

2.2. Latent states, ambiguity, and independent rate

An *encoding system* for a fact F is a finite collection of locations $\{L_1, \dots, L_n\}$ whose current values jointly determine the observation transcript available to a resolver. For a system state x , the *ambiguity class* is the set of latent values still compatible with that transcript; when the ambiguity class has size greater than one, the transcript alone does not identify a unique authoritative value. In this single-fact setting, latent-state ambiguity and value ambiguity coincide because the latent state is just the fact value. In the later multi-fact extension, ambiguity is instead over full latent tuples compatible with the partial observations. A *fact* is an atomic semantic attribute of the represented object that can vary independently of other attributes, and a fact is *structural* if the locations encoding it are fixed when the underlying object, declaration, or artifact is created.

The admissible edit set $E(C)$ is part of the primitive specification of an encoding system C . It records which edit events or edit sequences are considered reachable by the model. The paper does not derive $E(C)$ from a particular concurrency policy, protocol, or consensus mechanism; those are downstream instantiations. Independence and independent rate are therefore always defined *relative to the chosen admissible edit model*.

Definition 2.1 (Independent Location). Let $E(C)$ denote the admissible edit set of encoding system C . Two locations encoding the same fact are *independent relative to $E(C)$* if some admissible edit sequence in $E(C)$ can change one without forcing the other to match it, equivalently if some admissible edit sequence reaches a state in which the two locations disagree.

Definition 2.2 (Derived Location). A location is *derived* from a source location if updating the source determines the derived value without an additional manual edit.

Definition 2.3 (Independent Rate (Degrees of Freedom)). The *independent rate* of encoding system C for fact F is the number of pairwise independent source locations encoding F under the admissible edit set $E(C)$. We write $\text{DOF}(C, F)$ for this quantity. In the single-fact model, the remaining encoding locations are treated as derived views of those independent sources.

Definition 2.4 (Structural Integrity). An encoding system has *structural integrity* for fact F if every reachable state is coherent, so no reachable system state presents mutually incompatible encodings of F .

Definition 2.5 (Integrity Violation). A reachable *integrity violation* is a reachable incoherent state for fact F .

2.3. Zero-Incoherence Threshold

Remark. At independent rate 1, all locations are derived from a single source, so disagreement is unreachable. At independent rate above 1, admissible edits can assign different values to two independent locations, producing a reachable disagreement state.⁵

Definition 2.6 (Zero-Incoherence Threshold). The *zero-incoherence threshold* is the largest independent rate for which incoherent states are unreachable.

Corollary 2.7⁶ (Threshold Corollary). *For deterministic multi-location encodings, the zero-incoherence threshold equals 1.*

Proof. Achievability. Suppose $\text{DOF}(C, F) = 1$, and let L_s be the unique independent location. By Definition 2.3, every other encoding location is non-independent and is therefore treated in this model as a derived view of L_s . By Definition 2.2, updating L_s determines the value of each such derived

⁵ : COH1-2

⁶ : RAT1, CAP2-3

location without an additional manual edit. Hence all encoding locations agree in every reachable state, so zero incoherence is achieved.

Converse. Suppose $\text{DOF}(C, F) = k > 1$. By Definition 2.1, there exist at least two independent locations L_1, L_2 and an admissible edit sequence that can make them disagree. Choose values $v_1 \neq v_2$ and apply edits setting $L_1 \leftarrow v_1$ and then $L_2 \leftarrow v_2$. By independence, the second edit does not force L_1 to change. The resulting reachable state satisfies $\text{value}(L_1) \neq \text{value}(L_2)$ and is therefore incoherent.

Thus zero-error recovery is possible exactly at independent rate 1, so the zero-incoherence threshold equals 1. ■

This threshold identifies the unique zero-error corner of the deterministic model. Above that boundary, integrity violations are reachable by construction. The next section develops the finite converse family for the surviving ambiguity classes.

3. Unified Finite Converse

Theorem 3.1⁷ (Pair-injectivity characterization). *Let X range over a finite latent alphabet, let Y be the deterministic observation transcript, and let T be a finite auxiliary tag. Zero-error recovery from (Y, T) is possible if and only if the pair map $x \mapsto (Y(x), T(x))$ is injective on the ambiguity class under consideration.*

Proof. Let $\phi(x) := (Y(x), T(x))$.

Necessity. If ϕ is not injective, then there exist distinct latent states $x_1 \neq x_2$ with $\phi(x_1) = \phi(x_2)$. Any deterministic decoder D therefore receives the same input on both states and must satisfy $D(\phi(x_1)) = D(\phi(x_2))$. Since $x_1 \neq x_2$, the common output cannot equal both latent states, so D errs on at least one of them.

Sufficiency. If ϕ is injective, define a decoder on the image of ϕ by $D(y, t) = \phi^{-1}(y, t)$, with arbitrary extension off the image. Injectivity makes ϕ^{-1} single-valued on the image, and for every latent state x we then have $D(Y(x), T(x)) = x$. Thus zero-error recovery is possible. ■

⁷ : OBS1-2

Theorem 3.2⁸ (Confusability formulation). *Fix a deterministic observation transcript and an L -bit side-information tag. If K latent states induce the same observation transcript, then zero-error decoding on that ambiguity class requires K distinct tag outcomes. Equivalently, a K -way ambiguity class forms a confusability clique whose size cannot exceed the available tag alphabet.*

Proof. Apply Theorem 3.1 on an ambiguity class on which the observation is constant. Then injectivity of (Y, T) reduces to injectivity of the tag coordinate alone on that class. Since an L -bit tag provides at most 2^L outcomes, the clique size cannot exceed 2^L . ■

Corollary 3.3⁹ (Counting converse). *Theorem 3.2 specialized to a single K -way ambiguity class gives $K \leq 2^L$, i.e., at least $\log_2 K$ bits of side information.*

Proposition III.4 (Equivalent formulations). The pair-injectivity obstruction of Theorem 3.1 admits the following equivalent formulations on a finite source observed through a deterministic transcript with a finite auxiliary tag:

- *Global budget:* $K \leq OT$ for latent size K , observation alphabet size O , and tag alphabet size T .¹⁰
- *Fiber injectivity:* $|\mathcal{F}_y| \leq |\mathcal{T}|$ for each observation fiber $\mathcal{F}_y = \{x : Y(x) = y\}$.¹¹
- *Conditional entropy:* $H(X | Y) = \log_2 K$ on a K -way class with constant observation, and $H(T | Y) \geq \log_2 K$.¹²
- *Decoder output:* $H(\hat{X}) \leq H(Y, T) \leq H(X)$, with nonnegative gap to alphabet ceilings.¹³
- *Mass-weighted clique:* $\sum_{x \in S} p(x) \log_2 \frac{1}{p(x)} \leq h_2(p_S) + p_S \log_2 |\mathcal{T}|$ for a success-set clique S .¹⁴

⁸ : OBS5⁹ : CIA3¹⁰ : OBS4¹¹ : OBS3¹² : PRB47, PRB55¹³ : PRB73–74, PRB82–83, PRB90, PRB93–95¹⁴ : PRB55

- *Data processing:* $H(\kappa(Y, T)) \leq H(Y, T)$ for any deterministic coarsening κ .¹⁵

Proof sketch. Each formulation follows from Theorem 3.1 by restricting the injectivity condition to the relevant domain (fiber, clique, full source) and applying standard entropy identities. The confusability, counting, conditional-entropy, decoder-output, and finite-gap statements are therefore different normal forms of the same obstruction: a surviving K -way ambiguity class requires a budget of at least $\log_2 K$ bits to be resolved exactly.¹⁶

3.1. Finite-error extension

The main body of the paper studies zero error. The same deterministic finite model also supports a budgeted finite-error extension, which should be read as a deterministic finite analogue of the classical Fano line of argument rather than as a separate asymptotic coding theorem [7, 11–13].

Proposition 3.4¹⁷ (Budgeted finite-error converse). *Let P_e denote the decoder error probability on a finite source of size K , observed through a deterministic transcript alphabet of size O together with a tag alphabet of size T . Then the decoded-output entropy obeys*

$$H(\hat{X}) \leq h_2(P_e) + (1 - P_e) \log_2(OT) + P_e \log_2(K - 1).$$

Proof. Partition the source into success and failure events. On the success branch, the decoded output is constrained by the observation-tag budget and contributes at most $\log_2(OT)$. On the failure branch, the decoder can still output at most one of the remaining $K - 1$ alternatives. The Bernoulli split between the two branches contributes the binary-entropy term. Equivalently,

$$H(\hat{X}) = H(\hat{X} \mid \text{success/failure}) + H(\text{success/failure})$$

is bounded by combining the support bound on each branch with the binary entropy of the branch variable itself. ■

The zero-error theorems are the $P_e = 0$ boundary of this inequality. In that regime the success branch occupies all probability mass, the Bernoulli term vanishes, the failure contribution disappears, and the budget collapses back to the unified finite converse above.

¹⁵ : PRB68, PRB81, PRB83

¹⁶ : OBS1, OBS5, CIA3

¹⁷ : PRB85, PRB87

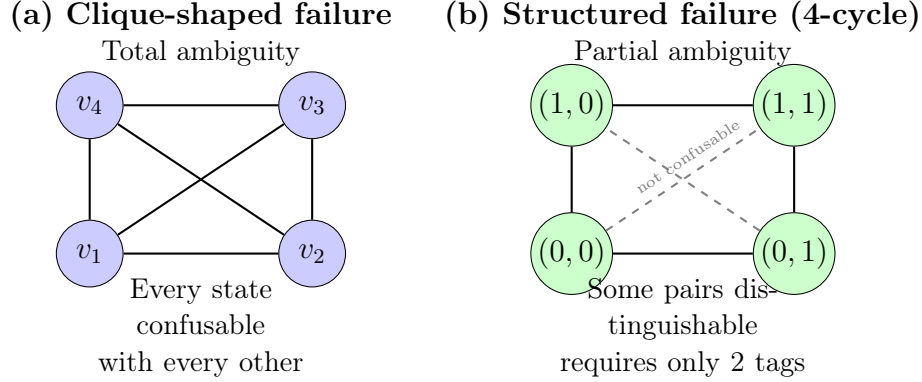


Figure 1: Two failure topologies. In (a), the observation provides no discrimination, so the confusability graph is a clique: every state is confusable with every other, and exact recovery requires a tag for each state. In (b), partial observations provide some discrimination: opposite corners are distinguishable even though adjacent states are not. The 4-cycle is 2-colorable, so exact recovery requires only 2 tags. This structural difference is invisible to the unit-rate threshold alone.

4. Graph Characterization of Partial Views

4.1. The Binary Square Witness

Consider a system with two binary facts $(x_1, x_2) \in \{0, 1\}^2$, giving four latent states:

$$(0, 0), \quad (0, 1), \quad (1, 0), \quad (1, 1).$$

View 1 reveals x_1 and View 2 reveals x_2 .

Confusability structure.. Two states are confusable when at least one view fails to separate them. View 1 reveals x_1 , so $(0, 0) \sim (0, 1)$ and $(1, 0) \sim (1, 1)$. View 2 reveals x_2 , so $(0, 0) \sim (1, 0)$ and $(0, 1) \sim (1, 1)$. The confusability graph is therefore a 4-cycle:

$$(0, 0) \sim (0, 1) \sim (1, 1) \sim (1, 0) \sim (0, 0),$$

where opposite corners are not adjacent.

Exact recovery.. The graph is 2-colorable (e.g., color by parity $c(x_1, x_2) = x_1 \oplus x_2$), so exact recovery is possible with a 2-ary auxiliary tag. With one tag, the best exact success probability under the uniform source is $1/2$.

Coding-theoretic interpretation. The parity coloring $c(x_1, x_2) = x_1 \oplus x_2$ is a syndrome bit: with $T = 2$ we decode perfectly, with $T = 1$ we cannot

distinguish adjacent vertices. The confusability graph is the confusion graph of a code with parity-check $H = [1 \ 1]$.

Figure 1 contrasts this structured failure with clique-shaped failure. The theorems below generalize this to arbitrary multi-fact partial-view systems.

4.2. General Graph Characterization

Definition 4.1 (Coordinate-Subset View). For a d -fact latent tuple $x = (x_1, \dots, x_d)$ and a coordinate set $S \subseteq [d]$, the *coordinate-subset view* indexed by S is the projection

$$\pi_S(x) := x|_S.$$

Definition 4.2 (Admissible View Family). An *admissible view family* is a finite family $\mathcal{V} = \{V_1, \dots, V_m\}$ of coordinate subsets of $[d]$, where view V_ℓ reveals exactly the coordinates indexed by V_ℓ .

Definition 4.3 (Agreement Set). For latent tuples $x, y \in \mathcal{X}$, their *agreement set* is

$$\text{Agr}(x, y) := \{i \in [d] : x_i = y_i\}.$$

Definition 4.4 (Upward-Closed Family). A family $\mathcal{U} \subseteq 2^{[d]}$ is *upward-closed* if whenever $S \in \mathcal{U}$ and $S \subseteq T \subseteq [d]$, one also has $T \in \mathcal{U}$.

Definition 4.5 (Confusability Graph). Given latent state space $\mathcal{X}$ and admissible view family $\mathcal{V}$, the *confusability graph* is the graph

$$G_{\text{conf}} = (\mathcal{X}, E_{\text{conf}})$$

whose vertices are the latent states and whose edges are the unordered pairs $\{x, y\}$ with $x \neq y$ for which there exists an admissible view $V_\ell \in \mathcal{V}$ such that $\pi_{V_\ell}(x) = \pi_{V_\ell}(y)$. Equivalently, $\{x, y\} \in E_{\text{conf}}$ iff some admissible view is contained in $\text{Agr}(x, y)$.

Definition 4.6 (Confusability Oracle). The *confusability oracle* is the Boolean adjacency test

$$\text{Conf}(x, y) = \mathbf{1}[\{x, y\} \in E_{\text{conf}}].$$

Definition 4.7 (Agreement-Set Oracle). The *agreement-set oracle* returns $\text{Agr}(x, y)$. In the coordinate-view model, one may decide confusability by computing $\text{Agr}(x, y)$ and testing whether some admissible view is contained in it.

For d facts over alphabet size q , explicit graph materialization has q^d vertices and at most q^{2d} ordered state pairs, but adjacency is available through the confusability oracle or the equivalent agreement-set oracle. The oracle computation takes $O(d + \sum_{\ell} |V_{\ell}|)$ time by computing the agreement set and scanning the admissible views.¹⁸

The Lean development also formalizes a support-overlap extension in which each location may emit any observation from a finite support set, and two states are confusable when some admissible location has overlapping support on those states.¹⁹

In the exact coordinate-view model, confusability depends only on agreement sets. From any admissible view family we obtain the upward-closed family

$$\mathcal{U} := \{S \subseteq [d] : \exists \ell, V_{\ell} \subseteq S\},$$

and two distinct tuples are adjacent if and only if their agreement set lies in $\mathcal{U}$. Conversely, every upward-closed family arises from some coordinate-view architecture. When the alphabet has at least two symbols, every proper coordinate subset occurs as the agreement set of some distinct pair of tuples, and coordinatewise alphabet permutations preserve confusability and induce graph automorphisms.²⁰

Theorem 4.8²¹ (Exact recovery as graph colorability). *For a finite multi-fact latent tuple observed through a finite family of partial-coordinate views, exact zero-error recovery with a T -ary auxiliary tag exists if and only if the induced confusability graph is T -colorable [14, 15].*

Proof. Let G_{conf} be the confusability graph.

Necessity. Suppose exact recovery with T tags is possible. If two latent tuples are adjacent in G_{conf} , then by definition some allowed observation leaves them indistinguishable. Exact recovery therefore cannot assign them the same tag, because the joint observation-tag pair would then coincide on two distinct latent tuples and Theorem 3.1 would be violated. Hence the tag assignment is a proper T -coloring of G_{conf} .

Sufficiency. Suppose c is a proper T -coloring of G_{conf} . Define the auxiliary tag of a latent tuple to be its color. If two latent tuples produce the

¹⁸ : MFT113–117, MFT132–135 ¹⁹ : MFT110–112 ²⁰ : MFT118–131 ²¹ : MFT3–4, MFT20

same observation and the same color, then they are confusable and adjacent, contradicting proper colorability. Therefore the joint observation-color map is injective, and Theorem 3.1 yields an exact decoder. ■

For repeated copies, the full block confusability graph is identified with the n -fold strong power of the one-shot confusability graph.²²

Theorem 4.9²³ (Success-set characterization). *In the same model, exact decoding on a designated success set S is equivalent to T -colorability of the induced confusability subgraph on S .*

Proof. Restrict the argument of Theorem 4.8 to the success set. Exactness is required only on S , so the coloring condition is required only on confusable pairs inside S . ■

Theorem 4.10²⁴ (Exact finite-error success budget). *For each tag alphabet size $T > 0$, there is a largest success-set size*

$$M_T := \max\{|S| : S \text{ induces a } T\text{-colorable subgraph}\}.$$

An exact decoder with T tags exists on a success set S if and only if $|S| \leq M_T$ after replacing S by some T -colorable success set of size M_T . Under the uniform source, the optimal exact success probability is therefore $M_T/|\mathcal{X}|$.

Proof. Theorem 4.9 identifies exact success sets with induced T -colorable subgraphs. Since the latent alphabet is finite, a maximum-cardinality such set exists. The mechanized characterization exposes this optimum directly. In the binary four-cycle witness, the optimum at $T = 1$ is exactly two states, so the best uniform exact success probability is $2/4 = 1/2$. ■

The same characterization is also available in weighted form. For any finite source weight w on the latent alphabet, the maximum mass of a T -colorable induced subgraph is the exact finite success value under budget T .²⁵

²² : MFT34–35

²³ : MFT9

²⁴ : MFT14–15

²⁵ : MFT18–19, MFT23–24

Theorem 4.11²⁶ (First non-clique witness). *There exists a binary two-fact system with single-coordinate observation views whose confusability graph is not a clique. In that system, two tags suffice for exact recovery, one tag is impossible, and any one-tag decoder can be exact on at most half of the four latent states under the uniform source.*

Proof. The witness is the four-state binary square with one location revealing the first coordinate and the other revealing the second. States that agree on one coordinate are confusable through the corresponding location, but opposite corners are not confusable, so the graph is a 4-cycle rather than a clique. An explicit exact two-tag decoder is obtained by the parity coloring $c(x_1, x_2) = x_1 \oplus x_2$. With one tag, any exact success set must be an independent set of the cycle, hence has size at most two, yielding success probability at most $1/2$ under the uniform source. ■

Definition 4.12²⁷ (Strong Product). For graphs G and H , the *strong product* $G \boxtimes H$ has vertex set $V(G) \times V(H)$. Two distinct vertices (u, v) and (u', v') are adjacent exactly when either $u = u'$ and $v \sim v'$, or $u \sim u'$ and $v = v'$, or $u \sim u'$ and $v \sim v'$.

Theorem 4.13 (Block-composition law). *If two partial-observation systems are colorable with tag alphabets of sizes T_1 and T_2 , then their block-composed product system is colorable with a tag alphabet of size $T_1 T_2$.*

Proof. Take proper colorings of the two component confusability graphs and encode the pair of colors as one color in the product alphabet. If two product states are confusable, then some allowed product view fails to separate them. In the product system this means that either the first coordinates agree and the second coordinates are confusable, or the second coordinates agree and the first coordinates are confusable, or both coordinates are confusable. These are exactly the three adjacency cases in the strong product $G_1 \boxtimes G_2$. Hence every confusable product pair is separated by at least one component coloring, and therefore by the paired color as well. ■

The product confusability graph is exactly the strong product of the two component graphs.²⁸ Clique-based lower bounds propagate to products: if

²⁶ : MFT5–8, MFT10

²⁷ : MFT11–13, MFT16

²⁸ : MFT25–26

the first component contains a confusability clique of size c_1 and the second contains one of size c_2 , then the product contains a clique of size $c_1 c_2$.²⁹

Example family: iterated binary squares.. Let W be the binary square from Theorem 4.11. The m -fold product has confusability graph $C_4^{\boxtimes m}$ and exact tag budget $T_{\text{exact}}(W^{\boxtimes m}) = 2^m$. This gives an infinite family of non-clique examples on which the graph object, exact budget, and strong-product law interact nontrivially.³⁰

The next section studies asymptotic zero-error rates for these strong powers.

5. Block Composition, Strong Powers, and Asymptotic Capacity

5.1. Motivation: Repeated Composition

Under n -fold block composition, the confusability graph becomes its n -fold *strong power*: $G_n = G^{\boxtimes n}$. Unlike the clique case where ambiguity explodes, structured graphs yield controlled growth via supermultiplicative independent-set behavior. The theorem below quantifies the resulting asymptotic rate.

5.2. Block Composition Law

A genuine supermultiplicative block law holds:

$$\alpha_{m+n} \geq \alpha_m \alpha_n,$$

where α_n denotes the largest one-shot zero-error code size in the n -block composed system. This is the correct discrete precursor to Shannon-capacity analysis: the admissible code-size sequence is already supermultiplicative at the graph level.³¹

More sharply, the block-level graph itself factors correctly. Confusability between concatenated $(m+n)$ -block states is equivalent to adjacency in the strong product of the m -block and n -block confusability graphs, packaged formally as an explicit graph isomorphism. Thus the repeated-block system is not only bounded by strong-product arguments at the codebook level; it

²⁹ : MFT17

³⁰ : MFT11-13, MFT16-17, MFT29-30

³¹ : MFT22

is literally a strong-product graph-power object for the same induced failure map.³²

Strong-product quantification.. Block confusability requires one location tuple working *simultaneously* for both blocks. Iterating yields $\alpha_1^n \leq \alpha_n$.³³

Definition 5.1 (Shannon Capacity). For a finite graph G , its *Shannon capacity* is

$$\Theta(G) := \sup_{n \geq 1} \frac{1}{n} \log \alpha(G^{\boxtimes n}),$$

where α denotes independence number and $G^{\boxtimes n}$ is the n -fold strong power.

For the induced confusability graph, this is the lower asymptotic capacity envelope

$$C_* := \sup_{n \geq 1} \frac{\log \alpha_n}{n},$$

identified with the maximum independent-set size of the n -block confusability graph.³⁴

Strong-product lower bounds give monotonicity: $\alpha_m^k \leq \alpha_{km}$ and $\frac{\log \alpha_m}{m} \leq \frac{\log \alpha_{km}}{km}$.³⁵

Theorem 5.2³⁶ (Asymptotic Shannon-capacity theorem). *Let α_n denote the largest one-shot zero-error code size in the n -block composed system. Then the normalized block rates*

$$\frac{\log \alpha_n}{n}$$

converge to a real asymptotic Shannon-capacity value, and that value is equal to the supremum of the finite block-rate sequence.

Proof. The n -block confusability graph is $G_n = G^{\boxtimes n}$. For any graphs H, K , the Cartesian product of independent sets $I_H \times I_K$ is independent in $H \boxtimes K$, giving $\alpha(H \boxtimes K) \geq \alpha(H)\alpha(K)$. Iterating yields supermultiplicativity $\alpha_{m+n} \geq \alpha_m \alpha_n$, equivalent to subadditivity of $\{-\log \alpha_n\}$. Boundedness follows from

³² : MFT29-30, MFT38-39 ³³ : MFT21 ³⁴ : MFT31-32, MFT35-36 ³⁵ : MFT33 ³⁶ : MFT43-49, MFT56

$\alpha_n \leq |V(G)|^n$. Fekete’s lemma [16] gives

$$\lim_{n \rightarrow \infty} \frac{\log \alpha_n}{n} = \sup_{n \geq 1} \frac{\log \alpha_n}{n},$$

the asymptotic Shannon capacity $\Theta(G)$. ■

Running example: Binary square.. For the 4-cycle C_4 from Section 4.1, $\alpha(C_4) = 2$ and $\alpha(C_4^{\boxtimes n}) = 2^n$, giving normalized rate $\log 2$ and asymptotic Shannon capacity $\Theta(C_4) = \log 2$. This matches the iterated binary-square family: one-shot budget is 2, m -block budget grows as 2^m .

Relation to graph capacity theory.. This places the extended model in the classical zero-error graph-capacity setting. Confusability graphs arise from deterministic view families on latent tuples; the architecture determines the graph, which then determines recoverability. The clique-fiber regime recovers the cluster-graph equality case via transitivity of confusability.

6. Theta Upper Theory

Definition 6.1 (Lovász ϑ). For a finite graph G , let $\vartheta(G)$ denote the classical Lovász theta number, equivalently the common value of the standard orthonormal-representation, primal semidefinite, and dual formulations. Its asymptotic upper value is

$$\vartheta_\infty(G) := \inf_{n \geq 1} \frac{1}{n} \log \vartheta(G^{\boxtimes n}).$$

The complement-chromatic bound is the first upper bound. The complement of the strong-power confusability graph is identified with the coordinatewise “there exists an offending coordinate” graph on the complement, and a coloring of the one-shot complement graph lifts to a coloring of every strong power by coloring each coordinate separately. Consequently,

$$\alpha(G^{\boxtimes n}) \leq \chi(\overline{G})^n, \quad \frac{\log \alpha(G^{\boxtimes n})}{n} \leq \log \chi(\overline{G}),$$

and the asymptotic capacity is bounded above by $\log \chi(\overline{G})$.³⁷

³⁷ : MFT50-55

At the one-shot level, the same upper invariant is matched with standard witness, orthonormal-representation, primal-PSD, and dual-theta formulations by explicit feasible-object equivalences. Applying these one-shot bounds to strong powers yields subadditive upper-rate sequences, so Fekete’s lemma [16] gives asymptotic primal and dual upper values, each equal to the infimum of its finite power-rate bounds. The finite block rates are bounded above by either sequence, hence

$$\Theta(G) \leq \vartheta_\infty(G).$$

The formal development proves these one-shot equivalences and the resulting asymptotic primal/dual upper laws.³⁸

What remains open is the sharpness of this upper theory for the graph classes generated by the extended model, not the existence of standard primal–dual theta packaging.

7. Equality Characterization

Classification of the non-clique boundary. The remaining problem in this section is to characterize which upward-closed agreement-set families produce transitive confusability, and therefore trigger the cluster-graph equality collapse. This does not weaken the equality route established here. Trivial single-view families and the fiber-coherent/meet-witnessed families already give broad sufficient conditions under which the collapse is completely determined; these are precisely the dominant patterns in architectures governed by a canonical transcript fiber or a common refining subview. The unresolved part is the residual non-clique boundary beyond those structural patterns. The binary square remains the smallest witness that this residual boundary is nonempty. A plausible conjectural direction is that transitivity of G_{conf} is governed by intersection-closure properties of the underlying admissible view family $\mathcal{V}$.

The original clique-fiber regime is now isolated as an equality subclass rather than only a threshold corollary. If a graph is generated by a surjective label map $x \mapsto b(x)$ with adjacency exactly between distinct vertices in the same fiber, then one representative from each fiber forms an independent set of size $|\mathcal{B}|$, while the complement graph is colorable by the same label

³⁸ : MFT57–84

map using $|\mathcal{B}|$ colors. This equality for cluster-graph structure is classical; the mechanized development packages the model-specific export needed here into a restricted-class theorem:

$$\Theta(G) = \vartheta_\infty(G) = \log |\mathcal{B}|.$$

Transitivity of confusability exports the model-generated graph directly to the classical cluster-graph setting through the following mechanized implication:

$$\text{confusability is transitive} \implies G_{\text{conf}} = \text{Cluster}(\Pi_{\text{cc}}).$$

Hence, if confusability is transitive, the base confusability graph is exactly the cluster graph on connected components, and the model-specific equality theorem yields

$$\Theta(G_{\mathcal{V}}) = \vartheta_\infty(G_{\mathcal{V}}) = \log |\Pi_{\text{cc}}|,$$

where Π_{cc} is the connected-component partition of the base confusability graph. Between arbitrary base models and full fiber coherence, a checkable sufficient condition is that the view family be *meet-witnessed*: every pair of allowed views contains another allowed view inside their intersection. Under that condition, any two confusability witnesses can be pulled back to a common subview, so confusability is transitive and the same equality follows. Fiber coherence is then a stronger sufficient structural condition: if equality at one allowed location already fixes the full observation transcript, transitivity follows, the connected components are exactly the realized transcript fibers, and the equality sharpens further to

$$\Theta(G_{\mathcal{V}}) = \vartheta_\infty(G_{\mathcal{V}}) = \log |\mathcal{T}_{\text{real}}|.$$

At the witness level, this same collapse admits a local composition form: base confusability is the edge union of the single-view cluster graphs, and transitivity is equivalent to closure of those fixed-view witnesses under two-step composition through an intermediate state. This is the exact local criterion for the current equality mechanism: the collapse to a cluster graph is determined by how single-view witnesses compose, not by an opaque global coincidence of the full confusability graph. Thus the equality question is structural: it identifies when genuinely non-clique failure geometry collapses to exact cluster behavior.³⁹

³⁹ : MFT85-91, MFT100-102, MFT107-109, MFT136-137

Condition	Logical status	Structural consequence	Capacity value
Fiber coherence	Stronger sufficient condition	Connected components are realized transcript fibers	$\log \mathcal{T}_{\text{real}} $
Meet-witness	Sufficient for transitivity and collapse	Base confusability collapses to the component cluster graph	$\log \Pi_{\text{cc}} $
Transitivity	Sufficient collapse condition used here	$G_{\text{conf}} = \text{Cluster}(\Pi_{\text{cc}})$	$\log \Pi_{\text{cc}} $

Table 1: Hierarchy of sufficient conditions for the cluster-graph equality route.

Running example: Binary square. The binary square of Section 4.1 does *not* have transitive confusability: $(0,0) \sim (0,1)$ and $(0,1) \sim (1,1)$, but $(0,0) \not\sim (1,1)$. The 4-cycle is not a cluster graph, so the transitivity-based equality collapse does not apply. In this special case the classical graph invariants still satisfy $\Theta(C_4) = \vartheta_\infty(C_4) = \log 2$, but that coincidence no longer comes from the cluster-collapse mechanism isolated in this section. The binary square therefore marks the boundary of the paper’s equality route: it is the first non-clique witness, yet not a transitive one.

Hierarchy summary. Table 1 summarizes the hierarchy of sufficient conditions for the cluster-graph collapse.

8. Affine Dual: Coordinate Matroid

The resulting object is a representable matroid on fact indices. It is the coordinate-side dual of the same realized-state structure that generates the confusability graph, and it provides computationally tractable upper bounds on confusability and capacity. The tractability claim is representation-sensitive: throughout this section, the affine family $A = a_0 + V$ is assumed to be given by an explicit linear presentation of V , for example a basis or generator matrix over $\mathbb{F}_q$, together with the explicit coordinate-subset view family. Under that input model, the relevant quantity is just the rank of a restricted coordinate map and is computable by Gaussian elimination.

Warm-up example.. Before stating the general proposition, consider the binary square from Section 4.1. Here the realized state family is the full affine space $A = \mathbb{F}_2^2$, so we may take $a_0 = (0, 0)$ and direction space $V = \mathbb{F}_2^2$ with generator matrix

$$G = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

The coordinate functionals are $e_1 = [1 \ 0]$ and $e_2 = [0 \ 1]$. A single-coordinate view, say $S = \{1\}$, has rank $t(S) = 1$: it captures one of the two available dimensions, so it reveals exactly half of the informational dimension of the state space. This is the finite-affine meaning of the later matroid rank bound: the coordinate-side quantity $t(S)$ measures how much of the state survives the view, and in the affine regime it is obtained by ordinary matrix rank.

Proposition 8.1 (Affine fact-matroid specialization). *Assume the realized state family is affine over a field, so that the valid latent tuples form an affine translate of a linear subspace of the ambient fact space. For any fact set S and fact index i , semantic determination of i by S is equivalent to membership of the coordinate functional for i in the linear span of the coordinate functionals indexed by S . Consequently the fact indices carry a representable matroid: a fact set is independent exactly when its coordinate functionals are linearly independent, and the minimal determining fact sets are exactly the bases, all of common cardinality equal to the rank.*⁴⁰

Proof. Let the affine realized-state family be $A = a_0 + V$, where a_0 is a fixed origin and V is a linear subspace of the ambient fact space. Fix a fact set S and a fact index i .

By definition, fact i is semantically determined by S exactly when for all $x, x' \in A$, agreement on every coordinate in S forces agreement on coordinate i . Writing $d = x - x'$, this is equivalent to the statement that every direction vector $d \in V$ whose coordinates in S vanish also satisfies $d_i = 0$.

Let e_j denote the coordinate functional for fact j . The condition above says precisely that

$$V \cap \bigcap_{j \in S} \ker(e_j) \subseteq \ker(e_i).$$

In finite-dimensional linear algebra, this is equivalent to e_i lying in the span of the coordinate functionals $\{e_j : j \in S\}$. Thus semantic determination by

⁴⁰ : AFM1-5

S is exactly span membership of the corresponding coordinate functional.

The coordinate functionals therefore realize a representable matroid on fact indices. In that matroid, independence is linear independence of the coordinate functionals, bases are the maximal independent spanning sets, and every basis has the common rank cardinality. Since span corresponds exactly to semantic determination, the minimal determining fact sets are precisely those bases. ■

The matroid and the confusability graph are complementary objects induced by the same realized-state model. Under the common specialization to finite affine families with coordinate-projection views, the matroid rank provides *computationally tractable* upper bounds on confusability and capacity. For each coordinate set S the quantity $t(S)$ is the rank of the restricted coordinate map $\pi_S|_V$.⁴¹ The formalization proves the basic size bounds $t(S) \leq |S|$ and $t(S) \leq \dim V$, and shows that $t(S) = |S|$ on linearly independent coordinate families while $t(S)$ reaches the full ambient determining rank exactly on determining sets.⁴² Hence each $t(S)$ is computable by Gaussian elimination on an explicit presentation of V , and the bounds of Corollary 8.3 are computable in polynomial time. The matroid rank $t(S)$ therefore provides an explicit upper bound on the Shannon capacity of the induced confusability graph, bypassing the hardness of the general capacity computation. Computing Shannon capacity for general graphs is hard (the independence number is NP-hard, and even the Lovász- ϑ bound requires semidefinite programming [17, 18]). In contrast, once the affine family is explicitly presented, the relevant rank quantities reduce to standard linear algebra.

8.1. View-fiber dimensions and capacity

Let $A = a_0 + V$ with V an r -dimensional linear subspace over a finite field $\mathbb{F}_q$, and let admissible views be coordinate-subset projections. For a coordinate set S write $t(S)$ for the rank of the corresponding coordinate functionals on V .

Proposition 8.2⁴³ (View-fiber clique sizes). *Let A and $t(S)$ be as above. Each projection-fiber for view S is an affine subspace of dimension $r - t(S)$ and*

⁴¹ : AFM10-11

⁴² : AFM6-9, AFM12

⁴³ : MFT86

size $q^{r-t(S)}$. The single-view confusability graph G_S is therefore a disjoint union of $q^{t(S)}$ cliques each of size $q^{r-t(S)}$. In particular

$$\alpha(G_S) = q^{t(S)}, \quad \Theta(G_S) = t(S) \log q,$$

where α is the independence number and Θ is Shannon capacity (logarithms in the same base used elsewhere).

Proof. The projection onto coordinates S is an affine-linear map whose linear part restricted to V has rank $t(S)$. Its kernel is therefore a linear subspace of V of dimension $r - t(S)$, so every fiber is an affine translate of that kernel and has cardinality $q^{r-t(S)}$. The fibers partition A , and within each fiber every pair of states is confusable under S , so G_S is a disjoint union of equal-size cliques. The number of fibers equals $|A|/q^{r-t(S)} = q^{t(S)}$, so one can select one representative per fiber to obtain an independent set of size $q^{t(S)}$, whence $\alpha(G_S) = q^{t(S)}$. The block-power identity for such clustered graphs gives $\alpha(G_S^{\boxtimes n}) = q^{nt(S)}$, and normalization yields $\Theta(G_S) = t(S) \log q$. ■

Corollary 8.3⁴⁴ (Matroid ranks give capacity bounds). *Let G be the full confusability graph generated by a family of coordinate-subset views $\mathcal{S}$. Then*

$$\alpha(G) \leq \min_{S \in \mathcal{S}} q^{t(S)} \quad \text{and} \quad \Theta(G) \leq \min_{S \in \mathcal{S}} t(S) \log q.$$

Thus the matroid rank function $t(\cdot)$ on coordinate sets supplies explicit upper bounds on one-shot and asymptotic exact-recovery rates for the induced confusability graph.

Proof. An independent set for G must be independent in each single-view graph G_S , hence its size is at most $\alpha(G_S) = q^{t(S)}$ for every $S \in \mathcal{S}$. The inequalities follow from taking the minimum over $\mathcal{S}$ and applying the block-power argument from the proposition. ■

Proposition 8.4 (Matroid direct sum under block composition). *For affine families $A_1 = a_1 + V_1$ over $\mathbb{F}_q^{d_1}$ and $A_2 = a_2 + V_2$ over $\mathbb{F}_q^{d_2}$, the representable matroid on $[d_1 + d_2]$ induced by the product family $A_1 \times A_2$ is the direct sum of the component matroids. A coordinate set $I \subseteq [d_1 + d_2]$ is independent in the product matroid if and only if $I \cap [d_1]$ is independent in matroid 1 and its*

⁴⁴ : FM6, AFM6-12

restriction to the right coordinate block $\{d_1 + 1, \dots, d_1 + d_2\}$ is independent in matroid 2.⁴⁵

Proof. The direction space of the product family is $V_1 \times V_2$. The restricted coordinate projection $\pi_{S_1 \cup S_2}|_{V_1 \times V_2}$ decomposes as $\pi_{S_1}|_{V_1} \times \pi_{S_2}|_{V_2}$ because V_1 and V_2 occupy disjoint coordinate blocks. Linear independence of coordinate functionals on $V_1 \times V_2$ therefore decomposes into independent conditions on each block. ■

Corollary 8.5 (Rank additivity). *For coordinate sets $S_1 \subseteq [d_1]$ and $S_2 \subseteq [d_2]$,*

$$t_{\text{product}}(S_1 \cup S_2) = t_1(S_1) + t_2(S_2).$$

Consequently the matroid capacity bound decomposes additively under block composition:

$$\Theta(G_{\text{product}}) \leq \min_{S_1, S_2} (t_1(S_1) + t_2(S_2)) \log q.$$

46

Proof. The rank of a direct sum of linear maps is the sum of the ranks. Apply Corollary 8.3 to the product family. ■

Remark. The matroid rank $t(S)$ is the “informational dimension” captured by coordinates in S , directly analogous to the rank function in representable matroids studied in combinatorial optimization [19–21] and coding theory [7]. Here $t(S) = r$ iff S determines the entire state, and any admissible view containing a basis removes confusability entirely (the induced graph is edgeless). These finite-affine, coordinate-view statements show the matroid is directly applicable to the graph/capacity arc; extensions beyond this specialization are directions for future work.

Running example: Binary square. The binary square $\{0, 1\}^2$ from Section 4.1 is an affine space over $\mathbb{F}_2$ with $r = 2$. Each single-coordinate view has matroid rank $t(\{1\}) = t(\{2\}) = 1$, giving fiber size $2^{2-1} = 2$ and single-view capacity bound $\Theta(G_{\{i\}}) \leq 1 \cdot \log 2 = \log 2$. The full confusability graph (the 4-cycle) achieves this bound, so the matroid rank bound is tight in this case. The matroid viewpoint shows that the 4-cycle structure arises precisely because each view captures only half the informational dimension.

⁴⁵ : AFM18–19

⁴⁶ : AFM17

9. Unit-Rate Boundary Characterization

An encoding system has *unit independent rate* when $\text{DOF}(C, F) = 1$. By Corollary 2.7, this is exactly the regime in which every reachable state remains coherent. Derived views preserve visible multiplicity without increasing independent rate, so the single-source regime is the unique exact-consistency boundary.⁴⁷

When a nontrivial ambiguity class survives the observation transcript, Theorem 3.3 forces additional side information for exact recovery. If exact correctness is still demanded without that side information, the architecture must rely on additional independently accessible support, thereby leaving the rate-1 regime. Appendix A contains the detailed derivation lemma and the full boundary proof package.⁴⁸

10. Threshold and Rate-Complexity Corollaries

The boundary has a sharp cost interpretation. In the rate-1 regime, one manual update to the authoritative source suffices and the remaining views follow deterministically, so synchronization cost stays $O(1)$. Above the threshold, if a fact is maintained at n independently writable locations without zero-delay synchronization, restoring exact consistency requires $\Omega(n)$ interventions. This creates an unbounded cost gap between rate-1 architectures and systems with many independently writable replicas. The detailed cost model, the information-constrained lower bound, and the unbounded-gap theorem are deferred to Appendix A.⁴⁹

11. Zero-Error Realizability

A host system realizes the rate-1 regime when one location is authoritative and every secondary representation is maintained as a deterministic view. Verifiable decoding requires two capabilities:

1. **Zero-delay state synchronization:** updates to the authoritative source propagate to derived locations as part of the same atomic update event, with no reachable intermediate disagreement.

⁴⁷ : COH1-2, DER2, RAT1

⁴⁸ : CIA3, BND2, RED1

⁴⁹ : BND1-3, CIA4

2. **Structural side-information:** the decoder has access to topology metadata distinguishing authoritative sources from dependent nodes.

These two requirements are the entire realizability content of the main criterion; the appendix supplies the timing arguments and independence witnesses establishing their necessity.

Definition 11.1 (Verifiable Structural Integrity). An encoding system has *verifiable structural integrity* for structural facts if it both realizes structural integrity in reachable states and exposes enough host-native information to certify that this integrity-preserving dependence structure holds.

Corollary 11.2⁵⁰ (Zero-Error Realizability Criterion). *An observer can verifiably decode the single-source zero-error regime if and only if the system provides both zero-delay state synchronization and structural side-information.*

Proof. The detailed timing argument, the two necessity theorems, and the independence witnesses are deferred to Appendix [Appendix A](#). Combining those results gives the criterion. ■

Applied to representative host instantiations, this criterion separates systems in which propagation and provenance are both host-native from systems missing one or both capabilities. We instantiate the criterion across the programming-language, database, and dependency-management contexts summarized in Table [B.2](#). Two of the eleven representative channel families satisfy the criterion, and nine fail at least one necessary condition. Every entry is checked against the mechanized sufficiency conditions of Corollary [11.2](#).

12. Related Work

12.1. Zero-Error and Side-Information Lineage

Shannon’s zero-error framework and its graph-theoretic refinements by Körner and Lovász provide the closest conceptual starting point [\[1–3\]](#). Körner–Orlitsky’s zero-error synthesis is also a direct reference point for this lineage [\[4\]](#). Witsenhausen’s zero-error side-information problem is directly relevant because it studies exact recovery under side information through graph coloring and label budgets [\[5\]](#). Our one-shot coloring statements lie in that lineage. As in Witsenhausen’s formulation, an observation law induces

⁵⁰ : SOT1

the relevant confusability graph. The difference is that here the observation law is generated by a deterministic multi-fact tuple-space architecture with admissible coordinate-subset views, so the model itself constrains what graphs can arise.

Slepian–Wolf coding and classical entropy converses provide a useful comparison point [6, 7, 11, 12]. Unlike the stochastic side information in Witsenhausen’s setting, however, our model is strictly deterministic and constrained by the coordinate-projection architecture of the latent tuple space. Coding-for-computing, functional compression, and graph-entropy/characteristic-graph viewpoints are also relevant because they study deterministic decoder targets and coding questions once an underlying graph or function structure has been specified [2, 8, 22–24]. Recent zero-error function-compression work also sits in this neighborhood [25, 26].

Witsenhausen’s setting starts from a side-information law and studies the coding problem for the graph it induces. Our setting starts from a deterministic partial-view architecture on latent tuples, with observations obtained by admissible coordinate-subset projections, and asks what graph structure that architecture can generate and what recovery laws follow from it. In the exact full-tuple-space coordinate-view model analyzed here, the realizable confusability relations are precisely those determined by upward-closed families of coordinate-agreement sets. Characteristic-graph formalisms handle arbitrary deterministic functions of the source, whereas the coordinate-projection restriction used here also yields coordinatewise alphabet-permutation automorphisms, a monotone agreement-set graph class, and closure under block composition by strong product. The paper therefore does not claim a new theorem about arbitrary graphs; it identifies a deterministic partial-view model with a fully characterized model-generated graph class on the full labeled tuple space. What it does not attempt is the broader unlabeled-isomorphism classification, or the corresponding classifications for restricted realized-state families and stochastic support-overlap models.

In our setting the side information is deterministic rather than stochastic, but it plays the same operational role: exact recovery becomes impossible when the available observation/tag alphabet is too small to separate the remaining alternatives.

After a confusability graph is fixed, the colorability, strong-power, Shannon-capacity, and Lovász- ϑ machinery used later is classical. The novelty here is identifying transitivity of view-generated confusability as the model-side condition that produces the cluster-graph equality subclass,

together with meet-witnessing and fiber coherence as structural sufficient conditions. On the upper-bound side, this follows the same classical-to-modern arc that includes Haemers-type bounds and asymptotic-spectrum viewpoints [27–29].

12.2. Consistency-Constrained Storage and Interaction

Consistency-constrained storage problems, including multi-version coding and related distributed-storage converses, show that correctness requirements impose unavoidable information costs [30]. The consistency literature establishes fundamental tradeoffs between consistency, availability, and partition tolerance [31–33], while correctness models such as linearizability specify formal guarantees for concurrent operations [34]. Here the object under study is a modifiable encoding architecture, and the main question is the exact-consistency cost of multiple independently writable representations of one latent fact.

12.3. Computational Realizations

Reflection, metadata systems, and maintenance mechanisms in host platforms provide examples of how the boundary-case realizability conditions can be instantiated in practice [35, 36]. These examples are secondary to the paper’s main zero-error and graph-capacity contribution.

12.4. Formal Verification

The Lean 4 formalization places the work in the tradition of mechanized mathematics and verified theory development [9, 10, 37].

13. Conclusion

This framework identifies the coordinate-projection architecture itself as the primary determinant of zero-error recoverability. In the full coordinate-view model, the realizable confusability relations are exactly the upward-closed agreement-set families; exact recovery with a T -ary auxiliary tag is equivalent to T -colorability; block composition lifts the one-shot graph to strong powers; and the normalized block rates converge to the Shannon capacity of the induced graph. The same graph carries the standard Lovász- ϑ

upper theory, and transitivity of confusability gives a model-side route to the cluster-graph equality case.⁵¹

Under an affine restriction on the realized state family, the coordinate side carries a representable matroid whose rank gives tractable upper bounds on one-shot and asymptotic recoverability. The matroid composes by direct sum under block composition, paralleling the strong-product law on the graph side. The same framework isolates the unit-rate boundary as the unique regime of verifiable structural integrity: for structural facts, exact realizability requires both zero-delay synchronization and structural side-information. Appendix [Appendix B](#) records representative model instantiations of that criterion.⁵²

Several questions remain open. The paper does not classify which upward-closed agreement-set families yield transitive confusability, nor the model-generated graph class up to unlabeled isomorphism. The restricted-state and support-overlap extensions are formalized but not developed textually, and the sharpness of the Lovász- ϑ upper theory on the non-clique subclass is unresolved. The archived Zenodo record for the Lean 4 formalization and submission source bundle is cited in [\[38\]](#).

Data and Code Availability

The Lean 4 formalization, supplementary proofs, manuscript PDF, and submission source bundle supporting the cited theorem handles are archived in the Zenodo record cited in [\[38\]](#). No separate experimental dataset is associated with this theoretical study.

Declaration of Generative AI and AI-assisted technologies in the writing process

Generative AI tools (including Codex, Claude Code, Augment, Kilo, and OpenCode) were used throughout this manuscript, across all sections (Abstract, Introduction, theoretical development, proof sketches, applications, conclusion, and appendix) and across all stages from initial drafting to final revision. The tools were used for boilerplate generation, prose and notation refinement, $\text{\LaTeX}$ /structure cleanup, translation of informal proof ideas into

⁵¹ : MFT3, MFT20, MFT35, MFT48, MFT69, MFT96, MFT102, MFT118–131
COH1–2, RAT1, SOT1

⁵² : AFM1–19,

candidate formal artifacts (Lean/L^AT_EX), and repeated adversarial reviewer-style critique passes to identify blind spots and clarity gaps.

The author retained full intellectual and editorial control, including problem selection, theorem statements, assumptions, novelty framing, acceptance criteria, and final inclusion/exclusion decisions. No technical claim was accepted solely from AI output. Formal claims reported as machine-verified were admitted only after Lean verification (no `sorry` in cited modules) and direct author review; Lean was used as an integrity gate for responsible AI-assisted research. The author is solely responsible for all statements, citations, and conclusions.

References

- [1] C. E. Shannon, Zero-error capacity of a noisy channel, IRE Transactions on Information Theory 2 (3) (1956) 8–19. [doi:10.1109/TIT.1956.1056798](https://doi.org/10.1109/TIT.1956.1056798).
- [2] J. Körner, Coding of an information source having ambiguous alphabet and the entropy of graphs, Transactions of the 6th Prague Conference on Information Theory (1973) 411–425.
- [3] L. Lovász, On the shannon capacity of a graph, IEEE Transactions on Information Theory 25 (1) (1979) 1–7. [doi:10.1109/TIT.1979.1055985](https://doi.org/10.1109/TIT.1979.1055985).
- [4] J. Körner, A. Orlitsky, Zero-error information theory, IEEE Transactions on Information Theory 44 (6) (1998) 2207–2229. [doi:10.1109/18.720537](https://doi.org/10.1109/18.720537).
- [5] H. S. Witsenhausen, The zero-error side information problem and chromatic numbers, IEEE Transactions on Information Theory 22 (5) (1976) 592–593.
- [6] D. Slepian, J. K. Wolf, Noiseless coding of correlated information sources, IEEE Transactions on Information Theory 19 (4) (1973) 471–480. [doi:10.1109/TIT.1973.1055037](https://doi.org/10.1109/TIT.1973.1055037).
- [7] T. M. Cover, J. A. Thomas, Elements of Information Theory, 2nd Edition, Wiley-Interscience, 2006. [doi:10.1002/047174882X](https://doi.org/10.1002/047174882X).

- [8] N. Alon, A. Orlitsky, Source coding and graph entropies, *IEEE Transactions on Information Theory* 42 (5) (1996) 1329–1339. [doi:10.1109/18.532875](https://doi.org/10.1109/18.532875).
- [9] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in: *Automated Deduction – CADE 28*, Springer, 2021, pp. 625–635. [doi:10.1007/978-3-030-79876-5_37](https://doi.org/10.1007/978-3-030-79876-5_37).
- [10] The mathlib Community, The Lean mathematical library, in: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2020, pp. 367–381. [doi:10.1145/3372885.3373824](https://doi.org/10.1145/3372885.3373824).
- [11] R. M. Fano, *Transmission of Information: A Statistical Theory of Communications*, MIT Press, Cambridge, MA, 1961.
- [12] H. S. Witsenhausen, A. D. Wyner, A conditional entropy bound for a pair of discrete random variables, *IEEE Transactions on Information Theory* 21 (5) (1975) 493–501.
- [13] B. G. Kelly, A. B. Wagner, Improved source coding exponents via witsenhausen’s rate, *IEEE Transactions on Information Theory* 57 (9) (2011) 5615–5633. [doi:10.1109/TIT.2011.2162178](https://doi.org/10.1109/TIT.2011.2162178).
- [14] D. B. West, *Introduction to Graph Theory*, 2nd Edition, Prentice Hall, 2001.
- [15] R. Diestel, *Graph Theory*, 5th Edition, Springer, 2017. [doi:10.1007/978-3-662-53622-3](https://doi.org/10.1007/978-3-662-53622-3).
- [16] M. Fekete, Über die verteilung der wurzeln bei gewissen algebraischen gleichungen mit ganzzahligen koeffizienten, *Mathematische Zeitschrift* 17 (1) (1923) 228–249. [doi:10.1007/BF01204614](https://doi.org/10.1007/BF01204614).
- [17] J. Pékós, Computing the shannon capacity of a graph, *Optimization Letters* 5 (3) (2011) 421–429. [doi:10.1007/s11590-010-0199-7](https://doi.org/10.1007/s11590-010-0199-7).
- [18] M. Grötschel, L. Lovász, A. Schrijver, The ellipsoid method and its consequences in combinatorial optimization, *Combinatorica* 1 (2) (1981) 169–197. [doi:10.1007/BF02579276](https://doi.org/10.1007/BF02579276).

- [19] J. Oxley, *Matroid Theory*, 2nd Edition, Oxford University Press, 2011.
- [20] D. J. A. Welsh, *Matroid Theory*, Academic Press, 1976.
- [21] A. Recski, *Matroid Theory and Its Applications in Electric Network Theory and in Statics*, Springer, 1989.
- [22] A. Orlitsky, J. R. Roche, Coding for computing, *IEEE Transactions on Information Theory* 47 (3) (2001) 903–917. [doi:10.1109/18.915643](#).
- [23] V. Doshi, D. Shah, M. Médard, M. Effros, Functional compression through graph coloring, *IEEE Transactions on Information Theory* 56 (8) (2010) 3901–3917. [doi:10.1109/TIT.2010.2050835](#).
- [24] G. Simonyi, Graph entropy: A survey (2001) 399–441 [doi:10.1007/3-540-44986-5_13](#).
- [25] X. Guang, R. Zhang, Zero-error distributed compression of binary arithmetic sum, *IEEE Transactions on Information Theory* 70 (5) (2024) 3100–3117. [doi:10.1109/TIT.2023.3319976](#).
- [26] Y. Liu, L. Ong, S. Johnson, J. Kliever, P. Sadeghi, P. L. Yeoh, Information leakage in zero-error source coding: A graph-theoretic perspective, in: 2021 IEEE International Symposium on Information Theory (ISIT), 2021, pp. 2590–2595. [doi:10.1109/ISIT45174.2021.9517778](#).
- [27] C. Haemers, An upper bound for the shannon capacity of a graph, in: *Algebraic Methods in Graph Theory*, Colloquia Mathematica Societatis János Bolyai, Vol. 25, 1978, pp. 267–272.
- [28] J. Zuiddam, The asymptotic spectrum of graphs and the shannon capacity, *Combinatorica* 39 (5) (2019) 1173–1184. [doi:10.1007/s00493-019-3992-5](#).
- [29] D. de Boer, P. Buys, J. Zuiddam, The asymptotic spectrum distance, graph limits, and the shannon capacity (2024). [arXiv:2404.16763](#), [doi:10.48550/arXiv.2404.16763](#).
- [30] K. V. Rashmi, N. B. Shah, K. Ramchandran, D. Gu, Multi-version coding: An information-theoretic perspective of consistent distributed storage, *IEEE Transactions on Information Theory* 63 (6) (2017) 4111–4128. [doi:10.1109/TIT.2017.2695219](#).

- [31] E. A. Brewer, Towards robust distributed systems, in: Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing, ACM, 2000, keynote introducing CAP conjecture: Consistency, Availability, Partition tolerance—pick two.
- [32] S. Gilbert, N. Lynch, Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services, ACM SIGACT News 33 (2) (2002) 51–59, formal proof of CAP theorem. [doi:10.1145/564585.564601](https://doi.org/10.1145/564585.564601).
- [33] J. Gray, Notes on data base operating systems, in: Operating Systems: An Advanced Course, Springer, 1978, pp. 393–481. [doi:10.1007/3-540-08755-9_9](https://doi.org/10.1007/3-540-08755-9_9).
- [34] M. Herlihy, J. M. Wing, Linearizability: A correctness condition for concurrent objects, in: Proceedings of the 11th ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing, 1990, pp. 139–150. [doi:10.1145/214548.214562](https://doi.org/10.1145/214548.214562).
- [35] G. Kiczales, J. des Rivières, D. G. Bobrow, The Art of the Metaobject Protocol, MIT Press, 1991.
- [36] B. C. Smith, Reflection and semantics in lisp, in: Proceedings of the 11th ACM SIGACT-SIGPLAN symposium on Principles of programming languages, 1984, pp. 23–35. [doi:10.1145/800017.800513](https://doi.org/10.1145/800017.800513).
- [37] X. Leroy, Formal verification of a realistic compiler, Communications of the ACM 52 (7) (2009) 107–115. [doi:10.1145/1538788.1538814](https://doi.org/10.1145/1538788.1538814).
- [38] T. Simas, [Exact recovery under deterministic partial views: Confusability graphs, strong powers, and capacity](https://zenodo.org/record/1935407), Zenodo, supplementary Lean 4 formalization, manuscript PDF, and source bundle. (2026). [doi:10.5281/zenodo.19354075](https://doi.org/10.5281/zenodo.19354075).
URL <https://doi.org/10.5281/zenodo.19354075>
- [39] M. Teich, R. Hettinger, Y. Selivanov, PEP 487 – simpler customisation of class creation, Python Enhancement Proposals, online: peps.python.org/pep-0487/ (2016).

- [40] Python Software Foundation, Python data model, Python 3 Documentation, [Online]. Available: docs.python.org/3/reference/datamodel.html (2025).
- [41] The PostgreSQL Global Development Group, Materialized views, PostgreSQL documentation, [Online]. Available: www.postgresql.org/docs/current/rules-materializedviews.html (2025).
- [42] The PostgreSQL Global Development Group, pg_matviews, PostgreSQL system catalog documentation, [Online]. Available: www.postgresql.org/docs/current/view-pg-matviews.html (2025).
- [43] The Rust Team, The Rust reference, Language reference, online: doc.rust-lang.org/reference/ (2024).
- [44] Microsoft, Decorators, TypeScript Handbook, [Online]. Available: www.typescriptlang.org/docs/handbook/decorators.html (2025).
- [45] MDN Web Docs, Classes, JavaScript reference, [Online]. Available: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes (2025).
- [46] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, G. Bierman, The Java Language Specification, Java SE 17 Edition, Oracle America, Inc., 2021, online: docs.oracle.com/javase/specs/.
- [47] The Go Authors, The Go programming language specification, Language specification, online: go.dev/ref/spec (2024).
- [48] npm, Inc., package-lock.json, npm CLI documentation, [Online]. Available: docs.npmjs.com/cli/v8/configuring-npm/package-lock-json/ (2026).
- [49] npm, Inc., npm explain, npm CLI documentation, [Online]. Available: docs.npmjs.com/cli/v7/commands/npm-explain/ (2026).
- [50] npm, Inc., npm ls, npm CLI documentation, [Online]. Available: docs.npmjs.com/cli/v7/commands/npm-ls/ (2026).
- [51] The Rust Project Developers, Cargo.toml vs cargo.lock, The Cargo Book, [Online]. Available: doc.rust-lang.org/cargo/guide/cargo-toml-vs-cargo-lock.html (2026).

- [52] The Rust Project Developers, cargo-tree, The Cargo Book, [Online]. Available: doc.rust-lang.org/cargo/commands/cargo-tree.html (2026).
- [53] The Rust Project Developers, cargo-metadata, The Cargo Book, [Online]. Available: doc.rust-lang.org/cargo/commands/cargo-metadata.html (2026).
- [54] Poetry Contributors, CLI, Poetry documentation, [Online]. Available: python-poetry.org/docs/cli/ (2026).
- [55] pnpm Contributors, pnpm install, pnpm documentation, [Online]. Available: pnpm.io/cli/install (2026).
- [56] pnpm Contributors, pnpm why, pnpm documentation, [Online]. Available: pnpm.io/cli/why (2026).
- [57] pnpm Contributors, pnpm list, pnpm documentation, [Online]. Available: pnpm.io/cli/list (2026).

Appendix A. Boundary and Realizability Proof Package

Appendix A.1. Derived Views and the Unit-Rate Boundary

An encoding system has *unit independent rate* when $\text{DOF}(C, F) = 1$. By Corollary 2.7, this is exactly the regime in which every reachable state remains coherent, and any rate above 1 admits reachable disagreement states. Thus unit rate is the unique exact-consistency boundary, and the later rate-complexity statements are its cost interpretation.⁵³

When ambiguity classes are trivial, no auxiliary information is needed and unit rate suffices. When a nontrivial ambiguity class survives the observation transcript, Theorem 3.3 shows that exact recovery requires a sufficiently large side-information alphabet. If that side information is unavailable but exact correctness is still demanded, additional independently accessible support is required. That is exactly what moves the system above unit rate and leads to the update-cost lower bounds below.⁵⁴

⁵³ : COH1-2, BND1, RAT1, RED1

⁵⁴ : CIA3, BND2, DER2

Definition Appendix A.1 (Derivation). Location L_{derived} is *derived from* L_{source} for fact F iff an update to L_{source} determines the value of L_{derived} without an additional manual edit.

Proposition Appendix A.2⁵⁵ (Derivation Preserves Coherence). *If L_{derived} is derived from L_{source} , then L_{derived} cannot diverge from L_{source} and does not contribute to DOF.*

Proof. By Definition Appendix A.1, the value at L_{derived} is determined by the value at L_{source} under admissible updates. There is therefore no reachable state in which the two locations carry incompatible values for the same fact. Since the derived location has no independently writable contribution, it does not increase DOF. ■

If all encodings of F except one are derived from that one, then Proposition Appendix A.2 leaves exactly one independent source, hence $\text{DOF}(C, F) = 1$ and exact consistency follows from Corollary 2.7.⁵⁶

Appendix A.2. Rate-Complexity Corollaries

Definition Appendix A.3 (Modification Cost Model). Let δ_F be a modification to fact F in encoding system C . The *synchronization cost* $S(C, \delta_F)$ is the minimum number of independent state interventions required to restore a zero-error joint state after applying δ_F . Deterministic dependent nodes contribute zero cost, as their state is compelled by the source update.

Theorem Appendix A.4⁵⁷ (Rate-1 Upper Bound). *If $\text{DOF}(C, F) = 1$, then*

$$S(C, \delta_F) = O(1).$$

Proof. When $\text{DOF}(C, F) = 1$, there is exactly one independently writable source location for F . Updating that location is one manual edit; every other representation of F is derived and is updated automatically. The number of manual edits therefore stays bounded by a constant independent of the number of visible encodings. ■

⁵⁵ : DER2

⁵⁶ : DER2, COH1

⁵⁷ : BND1

Theorem Appendix A.5⁵⁸ (Above-Threshold Lower Bound). *If fact F is encoded at n independent locations without zero-delay state synchronization, then*

$$S(C, \delta_F) = \Omega(n).$$

Proof. Each independent location can retain stale state unless acted upon directly. After a source update, every one of the n independent encodings can witness a distinct stale copy. Any protocol restoring zero-error consistency must therefore address each channel separately. No single intervention can synchronize two independently writable nodes, so the cost grows at least linearly in n . ■

Lemma Appendix A.6⁵⁹ (Information-Constrained DOF Lower Bound). *If the available side-information mechanism exposes at most 2^L distinguishable tags while the latent ambiguity class has size $K > 2^L$, then no zero-error resolver can identify the true state from those tags alone. Any architecture that still guarantees exact correctness must therefore rely on additional independently accessible discriminating support, moving the system above unit rate.*

Proof. Theorem 3.3 rules out exact recovery from the available tag budget. If exact correctness is nevertheless required, the architecture must supplement the insufficient side-information channel with additional independently accessible information. In the present model, that means leaving the rate-1 regime and paying the synchronization cost of Theorem Appendix A.5. ■

Theorem Appendix A.7⁶⁰ (Unbounded Gap). *The ratio between the synchronization cost of an architecture with n independent encodings and the synchronization cost of a rate-1 architecture diverges:*

$$\lim_{n \rightarrow \infty} \frac{S_{\text{DOF} > 1}(n)}{S_{\text{DOF} = 1}} = \infty.$$

Proof. By Theorem Appendix A.4, a rate-1 architecture has constant synchronization cost. By Theorem Appendix A.5, an architecture with n independent encodings incurs linear cost in n . The ratio therefore grows without bound. ■

⁵⁸ : BND2

⁵⁹ : CIA4

⁶⁰ : BND3

Appendix A.3. Realizability Details

A host system realizes the rate-1 regime when one location is authoritative and every secondary representation is maintained as a deterministic view. Verifiable decoding requires two capabilities:

1. **Zero-delay state synchronization:** updates to the authoritative source propagate to derived locations as part of the same atomic update event, with no reachable intermediate disagreement.
2. **Structural side-information:** the decoder has access to topology metadata distinguishing authoritative sources from dependent nodes.

To connect realizability to information available at verification time, fix a fact F and consider the alternatives that remain compatible with the available observations. These alternatives form the same zero-error obstruction analyzed in Sections 3–7: if the host cannot separate the surviving ambiguity class, exact verification requires additional structural side information.

Lemma Appendix A.8⁶¹ (Confusability Clique Bound). *If the confusability graph for F contains a clique of size k , then any zero-error side-information scheme distinguishing those k alternatives requires at least k distinct tags, equivalently at least $\log_2 k$ bits of side information.*

Proof. Within a k -clique, every pair of alternatives is confusable under the base observation. A zero-error tag assignment must therefore separate all k states. Distinct states cannot share a tag, so at least k tags are required. ■

Definition Appendix A.9 (Structural Fact). A fact F is *structural* if every location encoding F is fixed when the underlying object, declaration, or artifact is created. After that moment, the encoding can be read and compared, but not retroactively re-created without a new edit event.

Theorem Appendix A.10⁶² (Timing Constraint for Structural Derivation). *For structural facts, derivation must occur no later than the moment at which the relevant encoding locations become fixed.*

Proof. Let t_{fix} denote the time at which the structural encoding becomes fixed. A derivation performed strictly before t_{fix} cannot depend on the completed source value; a derivation performed strictly after t_{fix} cannot alter the

⁶¹ : OBS5

⁶² : REQ1, TRI1

already fixed structural representation. Therefore structural derivation must occur at t_{fix} or as part of the same creation/update event. ■

Definition Appendix A.11 (Zero-Delay State Synchronization). An encoding system has *zero-delay state synchronization* if every update to a source location is followed by updates to all locations derived from that source with no reachable intermediate state in which source and derived locations disagree. Equivalently, propagation is part of the same admissible update event for the purposes of reachable-state analysis.

Theorem Appendix A.12⁶³ (Zero-Delay Synchronization is Necessary for Unit Rate). *Achieving independent rate 1 for structural facts requires zero-delay state synchronization.*

Proof. By Theorem [Appendix A.10](#), structural derivation must occur at the moment the source-side structural encoding is fixed. If zero-delay synchronization is absent, then some derived location remains stale until a later manual action. During that interval the source and derived locations disagree, so the architecture does not realize exact consistency. Hence a verifiable rate-1 realization for structural facts requires zero-delay synchronization. ■

Definition Appendix A.13 (Structural Side-Information). An encoding system has *structural side-information* if it supports queries that reveal which locations encode a fact, which of those locations are authoritative, and which are derived.

Theorem Appendix A.14⁶⁴ (Structural Side-Information is Necessary for Verifiable Unit Rate). *Verifying that independent rate 1 holds requires structural side-information.*

Proof. To verify independent rate 1, one must enumerate the locations encoding the fact, identify the authoritative source, and confirm that every remaining location is derived from it. Without structural side-information, these checks cannot be completed from inside the host system. The architecture may be coherent on observed states, but the single-source claim is not verifiable. ■

⁶³ : REQ2

⁶⁴ : REQ3

Theorem Appendix A.15⁶⁵ (Requirements are Independent). 1. *An encoding system can have zero-delay state synchronization without structural side-information.*
 2. *An encoding system can have structural side-information without zero-delay state synchronization.*

Proof. For (1), consider a system that automatically refreshes all derived views after each source update but exposes no query interface for its derivation graph. Coherence may hold in reachable states, but the single-source claim is not inspectable. For (2), consider a system that records complete provenance metadata yet requires users to trigger propagation manually after each change. The derivation structure is visible, but stale views remain reachable. Hence neither property implies the other. ■

Proof of Corollary 11.2. Necessity follows from Theorems [Appendix A.12](#) and [Appendix A.14](#). For sufficiency, assume both properties hold. Zero-delay synchronization ensures that every derived location is updated as part of the same structural event as the source; structural side-information then certifies that all secondary encodings are in fact derived from that source. The architecture therefore realizes a verifiable single-source encoding, i.e., independent rate 1, which is exactly verifiable structural integrity in the sense of Definition [11.1](#).

Appendix B. Model Instantiations and Realizability Classification

The following table instantiates Corollary [11.2](#) on eleven representative deterministic partial-view channel families. Each row is a witness construction verifying or falsifying the two structural requirements against the mechanized sufficiency conditions. The classification demonstrates that the realizability criterion is non-vacuous and discriminating across representative host instantiations.

Table B.2: Zero-incoherence classification of representative channel witnesses in the deterministic partial-view model. The same theorem-level coordinates apply uniformly across programming-language runtimes, databases, and dependency managers. Each entry is verified against the mechanized sufficiency conditions of Corollary 11.2.

Channel	Instance	Class	Prop	Prov	Rate-1	Mechanism
Prog. languages	Python [39, 40]	C	yes	yes	yes	Class-definition hooks and runtime hierarchy introspection are host-native. ⁶⁶
Databases	Engine-maintained materialized view [41, 42]	C	yes	yes	yes	Engine maintains the derived relation and exposes catalog metadata. ⁶⁷
Prog. languages	Rust [43]	B	part.	no	no	Compile-time macro generation exists, but the compiled runtime artifact does not retain derivation provenance. ⁶⁸
Prog. languages	TypeScript / JavaScript [44, 45]	B	part.	no	no	Decorators can attach metadata, but type erasure and missing subclass provenance block exact verification. ⁶⁹
Prog. languages	Java [46]	A/B	no	no	no	Under the runtime-only host interpretation used here, annotations are external to the JVM runtime. ⁷⁰
Prog. languages	Go [47]	A/B	no	no	no	Reflection exists, but there are no definition-time hooks or host-native implementer registries. ⁷¹

Continued on next page

⁶⁶ : PYH1, PYI1 ⁶⁷ : DBV1-3 ⁶⁸ : RUH1, RUI1 ⁶⁹ : TSE1, TSN1 ⁷⁰ : JVA1 ⁷¹ : GOL1

Channel	Instance	Class	Prop	Prov	Rate- 1	Mechanism
Databases	External ETL copy / duplicate summary table	A/B	no	no	no	Synchronization is externalized, and provenance is no longer intrinsic to the host database. ⁷²
Deps.	npm [48–50]	A	no	yes	no	Lockfile (auxiliary tag) and query commands expose provenance, but manifest edits require explicit out-of-band operations. ⁷³
Deps.	Cargo [51–53]	A	no	yes	no	Resolved-dependency provenance is exposed, but the auxiliary tag (<code>Cargo.lock</code>) remains a distinct derived artifact. ⁷⁴
Deps.	Poetry [54]	A	no	yes	no	The resolved graph is queryable, but lock regeneration is an explicit step rather than part of the source edit itself. ⁷⁵
Deps.	pnpm [55–57]	A	no	yes	no	Provenance is queryable, but the lockfile remains separately maintained rather than automatically updated. ⁷⁶

⁷² : DBE1–3

⁷³ : NPM1–3

⁷⁴ : CRG1–3

⁷⁵ : POT1–3

⁷⁶ : PNM1–3